

Lab 4 – MNIST: Learning at Scale

Predictions

What I will change

- Manipulate data, size of hidden layer, initial random weights range, and learning rates to optimize the model to its best performance.
- Held fixed:
 - Train data size = 60,000
 - Validation data size = 10,000
 - Labels = 0 - 9
 - Activation functions = ReLU (hidden), Softmax (output)
 - Target Accuracy = 99.00%
 - Target Test Accuracy = 97.00%
 - Max Epoch = 20

What I predict will happen

Hidden Layer Size

Hidden layers with size 50 - 500 nodes will perform the best in reaching the targets in the fewest epochs.

Initial Weights

All positive weights will train the fastest.

Learning Rate

0.7 will need the fewest epochs.

Data Precision

Reducing the grey-scale precision will lower the test accuracy.

Why?

Hidden Layer Size

Hidden layers with size 50 - 500 nodes will perform the best in reaching the targets in the fewest epochs. A small layer (< 50) would not have enough capacity to reach 99% training

accuracy within 20 epochs. A large layer (> 500) will have far more parameters than the task needs, so it will memorize the training images and fall short of 97% test accuracy even if it reaches 99% on the training set.

Initial Weights

I predicted that all positive weights will train fastest because every ReLU unit will start active, so no neuron begins “dead” and every weight receives a gradient from the first batch. All negative weights will start to learn slowly because most hidden units output zero at first, but the biases and weights should shift over the first few epochs until the units become active and training catches up. Mixed weights would probably land somewhere in between.

Learning Rate

I believe that 0.7 will need the fewest epochs. Larger steps would cover more distance per update, and with a 20-epoch limit, the network that moves fastest toward the minimum should hit the targets first. 0.3 will be the slowest and may not reach 99% training accuracy in time.

Data Precision

I predict that reducing the grey scale precision will noticeably lower test accuracy. The grey values along the edges of each stroke come from anti-aliasing and carry the detail about the stroke's shape. Removing that detail would make the similar digits like (3 and 8), (2 and 5), or (4 and 9) harder to tell apart, overall reducing the accuracy of the model's prediction.

Task A

MNIST files use the IDX format, in which there is a big endian header followed by raw unsigned bytes.

Label files have an 8 byte header (number, count).

Image files have 16 byte header (number, count, rows, cols).

Skip the header and read the rest as unit8.

train: (60000, 28, 28) (60000,)

test: (10000, 28, 28) (10000,)

labels: [0 1 2 3 4 5 6 7 8 9]

pixels: 0 to 255

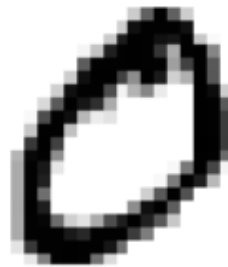
first labels: [5 0 4 1 9 2 1 3 1 4]

Task B

label: 5



label: 0



label: 4



Task C

Best Model (Cross-Entropy)

Hidden Nodes	Weight Range	Learning Rate	# of Epochs
300	(-0.05, 0.05)	0.300	5

Loss: 0.0327 | Accuracy: 99.01%

Val Loss: 0.0762 | Val Accuracy: 97.84%

Size of Hidden Nodes

Hidden Nodes	Weight Range	Learning Rate	# of Epochs	Loss, Accuracy	Val Loss, Val Accuracy
300	(-0.05, 0.05)	0.300	5	0.0327, 99.01%	0.0762, 97.84%
24 [< 50]	(-0.05, 0.05)	0.300	> 20	0.0625, 97.98%	0.1404, 96.65%
256 [(50, 500)]	(-0.05, 0.05)	0.300	6	0.0266, 99.18%	0.0613, 98.13%
1024 [> 500]	(-0.05, 0.05)	0.300	5	0.0259, 99.22%	0.0550, 98.19%

Initial Weight Range

Hidden Nodes	Weight Range	Learning Rate	# of Epochs	Loss, Accuracy	Val Loss, Val Accuracy
300	(-0.05, 0.05)	0.300	5	0.0327, 99.01%	0.0762, 97.84%
300	(0, 0.10) [+]	0.300	> 20	24.9034, 9.87%	24.9232, 9.80%
300	(-0.10, 0) [-]	0.300	> 20	2.302, 10.89%	2.3020, 10.28%

Learning Rates

Hidden Nodes	Weight Range	Learning Rate	# of Epochs	Loss, Accuracy	Val Loss, Val Accuracy
300	(-0.05, 0.05)	0.300	5	0.0327, 99.01%	0.0762, 97.84%
300	(-0.05, 0.05)	0.500	6	0.0231, 99.22%	0.0699, 98.12%
300	(-0.05, 0.05)	0.700	8	0.0256, 99.18%	0.0833, 98.04%

Additional Curation: Remove Precision of Data



seed 0 | grey | epochs: 5 | train: 99.01% | test: 97.84%

seed 0 | binary | epochs: 5 | train: 99.25% | test: 97.62%

seed 1 | grey | epochs: 6 | train: 99.25% | test: 97.93%

seed 1 | binary | epochs: 5 | train: 99.23% | test: 97.77%

seed 2 | grey | epochs: 5 | train: 99.01% | test: 97.97%

seed 2 | binary | epochs: 5 | train: 99.33% | test: 97.44%

seed 3 | grey | epochs: 6 | train: 99.20% | test: 98.30%

seed 3 | binary | epochs: 5 | train: 99.19% | test: 97.72%

seed 4 | grey | epochs: 6 | train: 99.23% | test: 97.99%

seed 4 | binary | epochs: 5 | train: 99.25% | test: 97.63%

grey average | epochs: 5.6 | train: 99.14% $\pm$ 0.11% | test: 98.01% $\pm$ 0.16%

binary average | epochs: 5.0 | train: 99.25% $\pm$ 0.04% | test: 97.64% $\pm$ 0.11%

Results over 5 seeds (training stops at the first epoch with $\geq 99\%$ train and $\geq 97\%$ test accuracy):

Seed	Grey epochs	Grey train	Grey test	Binary epochs	Binary train	Binary test
0	5	99.01%	97.84%	5	99.25%	97.62%
1	6	99.25%	97.93%	5	99.23%	97.77%
2	5	99.01%	97.97%	5	99.33%	97.44%
3	6	99.20%	98.30%	5	99.19%	97.72%
4	6	99.23%	97.99%	5	99.25%	97.63%
Average	5.6	99.14% $\pm$ 0.11%	98.01% $\pm$ 0.16%	5.0	99.25% $\pm$ 0.04%	97.64% $\pm$ 0.11%

Task D

epoch | train loss | test loss | train acc | test acc | test wrong

1 | 0.8805 | 0.4088 | 80.04% | 89.04% | 1096

5 | 0.2635 | 0.2421 | 92.52% | 92.99% | 701

10 | 0.1855 | 0.1778 | 94.78% | 94.92% | 508

25 | 0.0934 | 0.1025 | 97.49% | 96.81% | 319

50 | 0.0471 | 0.0727 | 98.80% | 97.79% | 221

75 | 0.0286 | 0.0633 | 99.42% | 98.00% | 200

100 | 0.0187 | 0.0597 | 99.72% | 98.09% | 191

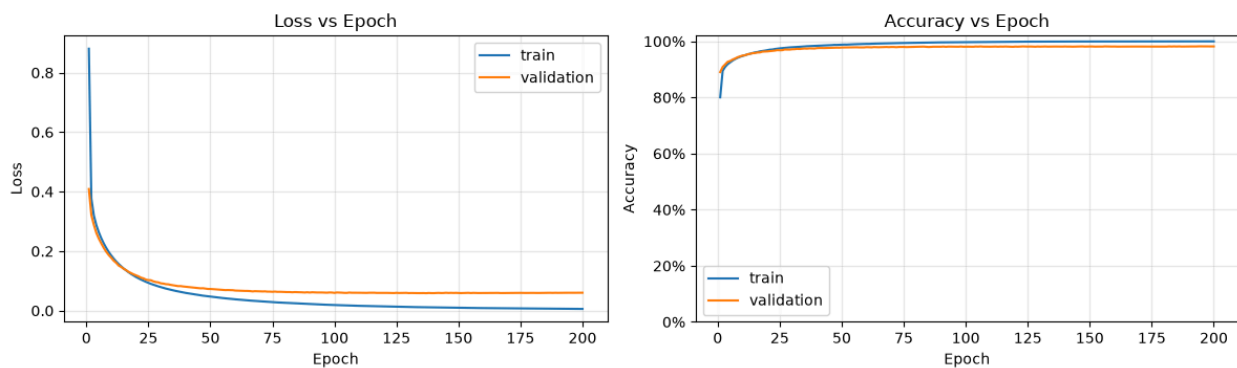
125	0.0131	0.0599	99.87%	98.12%	188
150	0.0096	0.0594	99.94%	98.16%	184
175	0.0073	0.0597	99.96%	98.15%	185
200	0.0057	0.0602	99.98%	98.18%	182

best test accuracy: 98.21% at epoch 194 (179 of 10000 wrong)

lowest test loss: 0.0587 at epoch 137

Training loss	Validation loss	Meaning
↓	↓	Learning well
↓	Flat	Converged
↓	↑	Overfitting
↑ or bouncing	Any	Unstable
Flat and high	Flat and high	Not learning

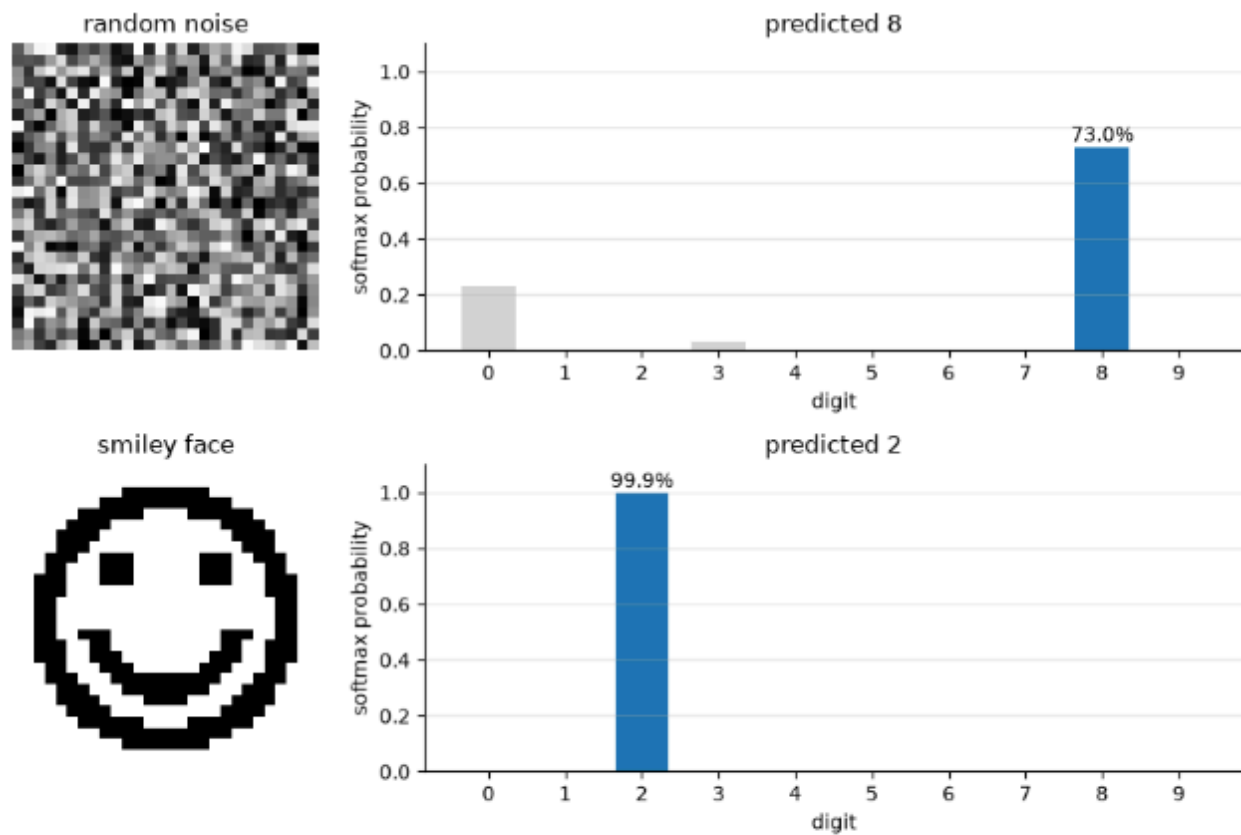
Training for 200 epochs (validation = test set)



Task E

Random noise: 5 with low or moderate confidence, maybe 30–60%

Smiley face: 0, the face is a big round outline, like a 0, with extra strokes inside



random noise -> 8 with 73.00% (runner-up 0 with 23.24%, probabilities sum to 1.000)

smiley face -> 2 with 99.87% (runner-up 3 with 0.09%, probabilities sum to 1.000)

Mechanism

Hidden Layer Size

The small network with 24 nodes never reached the targets because it did not have enough capacity and hit 97.98% training accuracy. With only 24 hidden nodes, the network has only 24 learned features to describe all ten digits and every handwriting style. The hard examples, like the digits that look similar to other digits, need features of their own, and there weren't enough to go around that. This was underfitting, as the training accuracy itself stalled, so the problem was fitting the data at all, not generalizing from it. The larger networks reached the targets faster. With 256 nodes, it took 6 epochs, 300 took 5 epochs, and 1024 took 5 with the best test accuracy of 98.19%. A wider layer starts with more random features, so more of them are already partly useful. Each update also adjusts more weights at once, so the output changes more per step. The 1024 node network did not overfit for this data. There was a gap between training and test accuracy ~ 1%.

Initial Weights

The mixed range of positives and negatives (0.05, 0.05) worked the best. Each hidden unit starts with a small pre-activation and each unit responds to a different random pattern of pixels. That gives the network varied features to build on. All-negative weights (-0.10, 0.00) left every hidden unit dead from the start. The pixels are non-negative and the biases start at zero. So, every pre-activation is ≤ 0 , and because ReLU outputs 0 for every image and ReLU derivative also produces 0, no gradient ever reaches the hidden weights. Since the hidden outputs are all 0, the output weights receive no gradient. The only parameters that can be learned are the output biases, so the network outputs the same distribution for every image. The loss of 2.302 confirms this as $\ln(10)$ is 2.302, which means that the loss of a uniform guess over ten classes is 2.302. All positive weights (0.00, 0.10) failed in the opposite way. The activation functions exploded. The first update changes each output weight by roughly $0.3 * 5 = 1.5$, about 30 times the size of the initial weights. The output logits then jump into thousands, and softmax saturates to near 100% confidence in a single class. Because every hidden unit is a positive sum of the same pixels, the units are also highly correlated. Each update overshoots, so prediction jumps from one class to another instead of learning to tell classes apart.

Learning Rate

The higher learning rate needed more epochs. The learning rate is supposed to scale the noise along the step. Early in training, big steps made faster progress, but after reaching 99% training accuracy, it depended on getting the last few hundred hard examples right, requiring fine adjustments. At 0.7, the steps are large enough to overshoot, fixing some examples while breaking others, so the network bounced around the minimum instead of settling into it.

Data Precision

Binarizing the pixels made the training faster and raised training accuracy, but dropped the testing accuracy. With the grey pixels, the model is able to identify which pixels are part of the stroke, not how dark they are. It also explains how the stroke curves, and exactly where the edges lie. Greyscale input changes smoothly as handwriting varies. If a writer's stroke sits slightly to the left, the grey values shift slightly, and the network's output shifts slightly too. With binary inputs, the same small change either does nothing or flips whole pixels from 0 to 1, which is a large, abrupt jump in the input. The binary network fit the training set more easily because its inputs had less variation to learn. But part of what it learned was the specific jagged, thresholded version of each training image, and those details don't carry over to new writers. As a result, the gap between training and test accuracy grew from 1.13% with greyscale to 1.61% with binary.

Reconciliation

I predicted

I predicted that a hidden layer of 50–500 nodes would reach the targets in the fewest epochs: a layer smaller than 50 would lack the capacity to reach 99% training accuracy, and a layer larger than 500 would overfit and fall short of 97% test accuracy. For the initial weights, I predicted that all-positive weights would train the fastest because every ReLU unit would start active, that all-negative weights would start slowly but recover as the weights and biases shifted, and that mixed weights would land in between. For the learning rate, I predicted that 0.7 would need the fewest epochs because larger steps cover more distance per update, and that 0.3 would be the slowest. Finally, I predicted that reducing the grey-scale precision would noticeably lower test accuracy, possibly below 97%, because the grey edges carry detail about each stroke's shape.

What actually happened

Hidden Layer Size

The small layer (24 nodes) failed as predicted, plateauing at 97.98% training accuracy. The large layer (1024 nodes) did not overfit: it reached the targets in 5 epochs, tied for the fastest, and had the best test accuracy (98.19%).

Initial Weights

Only the mixed weights trained, reaching the targets in 5 epochs. Both one-sided ranges stayed at chance level (about 10%). The all-negative network was stuck giving a uniform guess (loss = $\ln 10$), and the all-positive network predicted a single class for every image.

Learning Rate

The result was the opposite of my prediction: 0.3 took 5 epochs, 0.5 took 6, and 0.7 took 8.

Data Precision

Binarizing lowered test accuracy in all five seeds as predicted, but only by 0.37% (98.01% → 97.64%). It also reached the targets faster, with higher training accuracy.

What my mental was missing

Hidden Layer Size

I thought that the number of parameters as if it determined overfitting on its own. Overfitting also depends on how long the network trains. Memorizing individual training images only happens after the shared structure has been learned, much later than 20 epochs. I also missed that a wider layer is an advantage early in training. It starts with more random features, and more of them are already partly useful, so it learns faster per epoch.

Initial Weights

I only thought about whether ReLU units start "alive" or "dead." I missed two things. First was how the MNIST inputs are all non-negative. With zero biases, the sign of the weights

alone decides the sign of every pre-activation, for every unit and every image. That makes all-negative weights permanently fatal, not just slow. A dead unit has a gradient of exactly zero, so nothing can ever push it back into the active range. My idea that "the biases will shift" had no gradient to drive it. On the other hand, a unit can be too active, not just dead. With all-positive weights, every hidden unit adds up the ink with positive weights, so the activations were large and nearly identical across units. The first updates were far larger than the weights themselves, which saturated softmax. The network needs weights that are small, so the outputs stay in a range where gradients are useful, and varied in sign, so each unit detects a different pattern.

Learning Rate

Reaching 99% training accuracy depends on the last few hundred hard examples, and getting those right requires fine adjustments. A large step overshoots near the minimum, so as it fixes some examples while breaking others, so the network bounces around instead of settling. A large learning rate helps early in training but costs more at the end, and a target like 99% is decided at the end.

Data Precision

I was right that the grey levels carry real information, but I misjudged how much of a digit's identity depends on them. Most of what makes a 3 a 3 is which pixels belong to the stroke, and binarizing keeps that. The grey levels add finer detail that only decides the hardest, most ambiguous digits, so the drop was small. I also didn't consider that simpler inputs are easier to fit. Binary images have less variation, so the network reached 99% training accuracy sooner.